



赞同

分享

机器学习实战--KNN

王书成
python机器学习

关注

机器学习案例--KNN

给定一个[训练数据集⁺](#)，对新的输入实例，在训练数据集中找到与该实例最邻近的 k 个实例，这 k 个实例的多数属于某个类，就把该输入实例分为这个类。

项目案例1: 优化约会网站的配对效果

项目概述

海伦使用约会网站寻找约会对象。经过一段时间之后，她发现曾交往过三种类型的人: 不喜欢的人

魅力一般的人

极具魅力的人

她希望: 工作日与魅力一般的人约会

周末与极具魅力的人约会

不喜欢的人则直接排除掉

现在她收集到了一些约会网站未曾记录的数据信息，这更有助于匹配对象的归类。

[海伦⁺](#)把这些约会对象的数据存放在文本文件 `datingTestSet2.txt` 中，总共有 1000 行。海伦约会的对象主要包含以下 3 种特征：

- 每年获得的飞行常客里程数
- 玩视频游戏所耗时间百分比
- 每周消费的冰淇淋公升数

文本文件数据格式如下(最后一列为标签，3是极具魅力的人，2是魅力一般的人，1是不喜欢的人)：



26052	1.441871	0.805124	1
75136	13.147394	0.428964	1
38344	1.669788	0.134296	1

知乎 @王书成

第一步：从文本中读取数据

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif']=['SimHei'] #用来正常显示中文标签
plt.rcParams['axes.unicode_minus']=False #用来正常显示负号
def file2matrix(filename):
    train_x, train_y = [], []
    with open(filename, 'r') as f:
        for line in f.readlines():
            line = line.strip().split('\t')
            train_x.append(line[:3])
            train_y.append(int(line[-1]))

    train_x = np.array(train_x)
    train_x = train_x.astype(float)
    train_y = np.array(train_y)
    return train_x, train_y

x,y = file2matrix('datingTestSet2.txt')
print(x[:5])
```

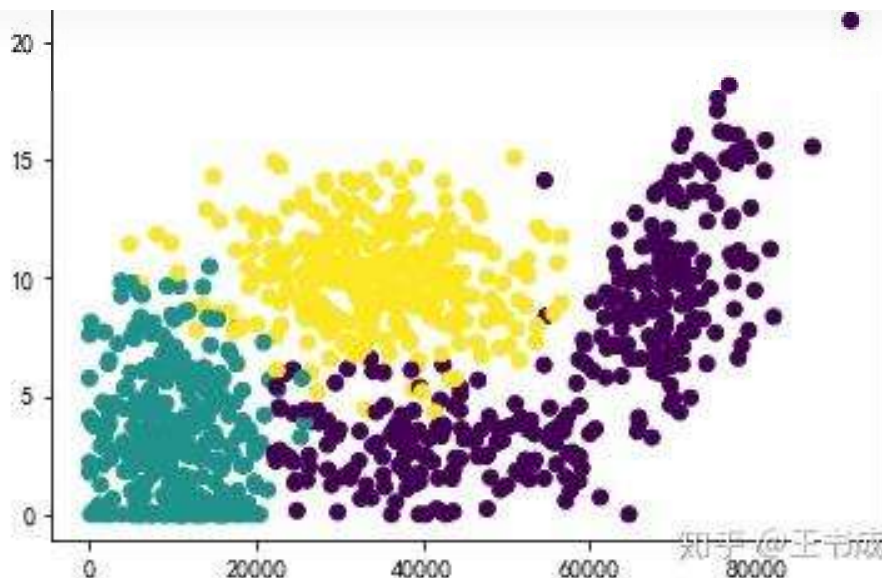
```
[[ 4.09200000e+04  8.32697600e+00  9.53952000e-01]
 [ 1.44880000e+04  7.15346900e+00  1.67390400e+00]
 [ 2.60520000e+04  1.44187100e+00  8.05124000e-01]
 [ 7.51360000e+04  1.31473940e+01  4.28964000e-01]
 [ 3.83440000e+04  1.66978800e+00  1.34296000e-01]]
```

第二步：分析数据

使用 Matplotlib 画二维散点图，

```
plt.scatter(x[:,0],x[:,1],c=y)
plt.title('第一列与第二列的散点图')
plt.show()
```

知乎

首发于
python从爬虫到机器学习

第三步：编写KNN

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split, cross_val_score  #划分数据 交叉验证

def knn0(x_train, y_train, x_test, y_test):
    knn = KNeighborsClassifier(5)
    knn.fit(x_train, y_train)
    return knn.score(x_test, y_test)

x_train, x_test, y_train, y_test = train_test_split(x, y)
knn0(x_train, y_train, x_test, y_test)

0.79200000000000004
```

对于上个代码块，每次运行的结果都不相同，因为每次训练的数据集都不相同，下面采用[十折交叉验证](#)的方法计算正确率

```
def knn1(x, y):
    knn = KNeighborsClassifier(5)
    scores = cross_val_score(knn, x, y, cv=10, scoring='accuracy') #cv选择每次测试折数
    return scores.mean()

knn1(x, y)

0.7882009789214216
```

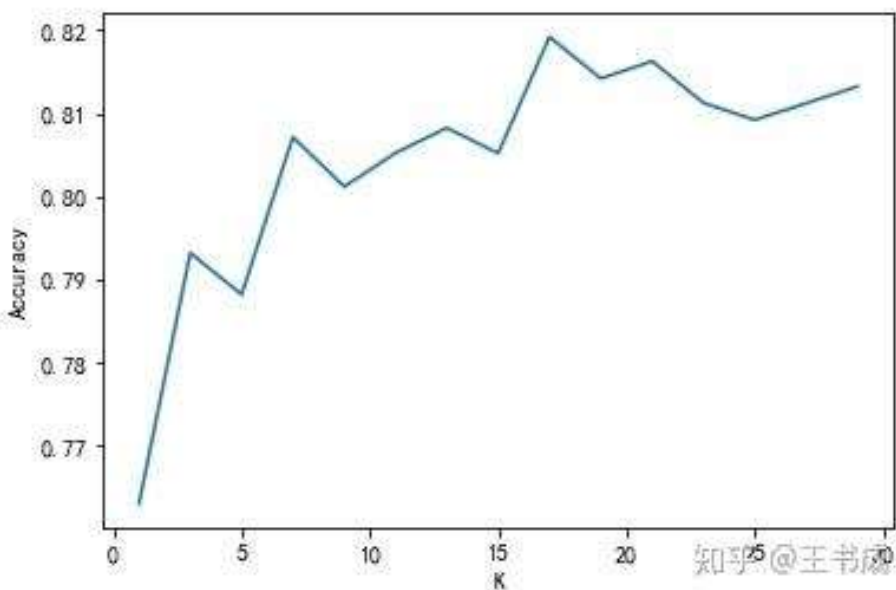
接下来看看不同数量的近邻对正确率的影响：

```
def knnTest(x, y):
    accr = []
    for i in range(1, 30, 2):
        knn = KNeighborsClassifier(i)
        scores = cross_val_score(knn, x, y, cv=10, scoring='accuracy') #cv选择每次测试折数
        accr.append(scores.mean())
    return accr

acc = knnTest(x, y)
plt.plot(1, accr)
```

知乎

首发于
python从爬虫到机器学习
plt.show()



可以发现当K=17时，正确率最高

第一列的数据比较大，对结果影响最大，需要进行归一化*处理：

```
from sklearn.preprocessing import MinMaxScaler

mm = MinMaxScaler()
xc = mm.fit_transform(x)

print(xc[:5])
[[ 0.44832535  0.39805139  0.56233353]
 [ 0.15873259  0.34195467  0.98724416]
 [ 0.28542943  0.06892523  0.47449629]
 [ 0.82320073  0.62848007  0.25248929]
 [ 0.42010233  0.07982027  0.0785783 ]]

def knn2(x, y):
    knn=KNeighborsClassifier(17)
    scores = cross_val_score(knn, x, y, cv=10, scoring='accuracy') #cv选择每次测试折数
    return scores.mean()

knn2(xc, y)
0.94911604101586633
```

当标准化之后，正确率得到了提高，就达到了94%

第四步，使用第三步的参数，编写分类器

```
def datingClassTest(test):
    x, y = file2matrix('datingTestSet2.txt')
    #归一化
    xmin = np.min(x,axis=0)
    xmax = np.max(x,axis=0)
    mm = MinMaxScaler()
    test = np.array(test)
    test
    test
```


知乎

首发于
python从爬虫到机器学习

```
# 1. 导入训练数据
train_labels = []
train_data = []
trainingFileList = os.listdir('trainingDigits') # load the training set
for item in trainingFileList:
    train_data.append(text2vector('trainingDigits/'+item))
    train_labels.append(int(item.split('_')[0]))
# 2. 导入测试数据
test_labels = []
test_data = []
testFileList = os.listdir('testDigits') # iterate through the test set
for item in testFileList:
    test_data.append(text2vector('testDigits/'+item))
    test_labels.append(int(item.split('_')[0]))
knn=KNeighborsClassifier(21)
knn.fit(train_data,train_labels)
return knn.score(test_data,test_labels)
```

```
handwritingClassTest()
0.97040169133192389
```

KNN 小结

KNN 是一个简单的**无显示学习过程**，**非泛化学习的监督学习模型**。在分类和回归中均有应用。简单来说：通过距离度量来计算**查询点**（query point）与每个训练数据点的距离，然后选出与查询点（query point）相近的K个最邻点（K nearest neighbors），使用分类决策来选出对应的标签来作为该查询点的标签。

KNN 三要素

- K的取值
- 距离度量
- 分类决策

发布于 2020-03-19 12:56

送礼物

还没有人送礼物，鼓励一下作者吧

内容所属专栏



python从爬虫到机器学习

订阅专栏

机器学习实战（书籍）

机器学习（Drew Conway, John Myles White 著）（书籍）

机器学习



理性发言，友善互动

知乎

首发于
python从爬虫到机器学习

还没有评论，发表第一个评论吧

推荐阅读

做图机器学习系统一年的研究总结（三）

一、前言-----
-----我是更新线-----
----- 我们
的系统开源啦！GitHub链接见这里
~ GitHub - quiver-team/torch-
quiver: PyTor...

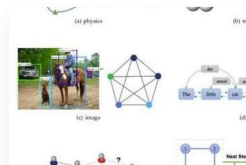
袁秀龙 发表于一亩三分地

NeurIPS21 | 从图模型到
GNN-机器学习中的消息传递...

Houye

从传统图引擎到GNN：计算图
和机器学习的演变

读芯术



2020年图机器学习的最

忆臻 发表于机器